

Helwan University
Faculty of Computers and Artificial Intelligence
Computer Science Department

EALPR SYSTEM

A graduation project dissertation by:

Mohamed Mahmoud Hanafy Yassin 20210827

Mahmoud Aly Mahmoud Ahmed 20210871

Gamal El-Din Ayman Abdel-Rahman 20210250

Zeyad Ameen Hussein Masoud 20210359

Rashad Samir Rashad Eid 20210331

Submitted in partial fulfilment of the requirements for the degree of Bachelor of
Science in Computers & Artificial Intelligence, at the Computer Science Department,
the Faculty of Computers & Artificial Intelligence, Helwan University

Supervised by:

[Dr. Mohammed El-Said]

June 2025

ACKNOWLEDGEMENT

**At first we thank ALLAH for giving us the
power to achieve this work, and for the
blessings from ALLAH to all of us,
El hamdullelah.**

**It is our pleasure to express our heartfelt
thanks to Dr. El-Said, Supervisor for our
graduation project, for his supervision and
guidance which enabled us to understand and develop
this project**

Abstract

Our application is a smart system designed to automate license plate recognition and streamline vehicle identification in real time. It supports traffic monitoring, parking management, and security systems by providing an intelligent way to detect and recognize car license plates from live video feeds and retrieve related driver data from a database.

The project addresses the increasing need for accurate, automated vehicle recognition systems by combining artificial intelligence, computer vision, and database integration. The application captures frames from a connected camera, identifies the location of the license plate, extracts the characters on the plate, and displays relevant information if the plate exists in the system's database.

Car Plate Recognition System: **AI-based Application**

The system consists of three primary phases:

- **Detection Phase**

A deep learning model is used to identify and locate license plates from a live video stream in real time. The plate region is automatically detected and extracted from the video feed.

- **Recognition Phase**

The detected plate is processed by a second model specialized in recognizing Arabic letters and numbers. These characters are interpreted and displayed in a readable form using Arabic text handling techniques.

- **Database Interaction Phase**

Once the plate content is recognized, the system checks it against an internal database. If a match is found, detailed information about the vehicle and the driver is retrieved and displayed through the application interface.

This solution offers a fast, efficient, and user-friendly way to monitor vehicles, verify license plates, and support smart infrastructure systems. It eliminates the need for manual entry and increases the speed and accuracy of vehicle identification.

Chapter 1: Introduction.....	7
1.1 Introduction.....	7
1.2 Motivation	8
1.3 Problem Definitions	8
1.4 Purpose	9
1.5 Scope	9
1.6 Overview.....	10
1.7 Similar Systems	10
Chapter 2: Background.....	13
2.1 introduction	13
2.2 problem statment.....	14
2.3 project goals.....	15
2.4 Related technologies.....	16
2.4.1 OpenCV.....	16
2.4.2 EasyOCR	17
2.4.3 MongoDB.....	17
2.5 Egyptian License Plates Recognition.....	18
2.6 Use Cases.....	19
Chapter 3: System Design.....	20
3.1 System Features	20
3.2 Usecase Diagram	22
3.3 Sequence Diagram.....	23
3.4 Activity diagram	24
3.5 Class Diagram.....	25
3 Chapter 4: System Implementation and results.....	26

4.1 Overview on Implementation steps.....	26
4.2 Future Work	32
4. Screenshots and Code Sample	35
Bibliography.....	46

Chapter 1: Introduction

Our idea focuses on helping authorities and smart systems automate the process of recognizing vehicle license plates to ensure security, efficiency, and speed in traffic monitoring environments.

1.1 Introduction

What if verifying vehicles could be done instantly and automatically? Our project aims to reduce the manual effort in identifying car plates by building a system that detects and reads license plates in real time using AI models, and matches them with a database to retrieve driver or vehicle data. This type of application is increasingly valuable in places like university entrances, smart gates, and security checkpoints.

The Car Plate Recognition System improves the accuracy and speed of vehicle identification. It reduces human error, enhances access control, and supports digital transformation in traffic and security systems.

1.2 Motivation

Today, many institutions—such as universities, gated communities, and smart cities—face challenges in monitoring and verifying vehicle access. Manual checks are time-consuming, error-prone, and not scalable.

The motivation behind our project is to automate this process using artificial intelligence. With the increasing availability of computer vision and smart camera systems, it is now possible to recognize license plates on the spot, read their content, and instantly check their existence in a secure database.

This application reduces the reliance on traditional methods and brings modern, AI-based automation to vehicle access systems. It also allows real-time alerts or data retrieval based on license plate recognition, which is especially helpful in controlled environments.

1.3 Problem Definitions

The main difficulties facing the purpose of the application are:

1. Manual inspection of vehicle plates is slow and inefficient, especially in busy or secured environments.
2. Human error in reading or recording plate numbers can lead to security risks or access control failures.
3. Lack of real-time systems that can automatically detect and recognize Arabic license plates with high accuracy.

4. The absence of an integrated solution that links recognized plate data with a local database to fetch relevant vehicle and driver details.

1.4 Purpose

The system assists security personnel and smart systems by automating the detection and recognition of vehicle license plates using a connected camera and artificial intelligence.

It helps identify vehicles in real time by processing a live camera feed, detecting car plates, recognizing the Arabic characters and digits, and matching them with a local database to retrieve and display vehicle or driver information instantly.

The application can also serve as a foundation for future integration with gates, alarms, and smart parking systems — enabling institutions to control access in a fast, secure, and modern way, while minimizing human involvement.

1.5 Scope

The system focuses on environments where vehicle identification is essential, such as university gates, secure facilities, parking areas, and smart access control systems.

It aims to provide a modern, automated solution to detect and recognize Egyptian car plates in real time. The system enhances security, improves efficiency, and reduces manual effort by linking

recognized plates with a database to instantly retrieve relevant driver or vehicle information.

The solution also opens possibilities for future expansion, such as integration with barriers, smart gates, or automated logging systems.

1.6 Overview

This application supports the growing need for intelligent vehicle monitoring systems, aligning with modern trends in smart cities, security automation, and real-time data processing.

It is designed to operate continuously and interactively by analyzing video feeds, identifying license plates, recognizing their content, and presenting associated information when available. The system can be deployed in any environment that requires rapid and accurate vehicle identification, reducing delays and minimizing human intervention.

By automating this process, the system improves operational flow and contributes to building a secure and efficient environment for organizations, campuses, and institutions.

1.7 Similar Systems

Several license plate recognition systems have been developed and deployed in recent years to meet the growing needs of smart cities, transportation authorities, and security sectors. These systems aim to automate the identification of vehicles by recognizing license plates from images or real-time video streams.

One of the earliest and most referenced systems is **OpenALPR**, which gained popularity for being open-source and later evolved into Rekor Scout. It provides plate recognition capabilities for a wide range of countries and was widely adopted in experimental and commercial setups. However, its application in regions like Egypt may be limited due to plate format differences, which often require model retraining or custom configuration.

Another well-known platform is **PlateRecognizer**, a cloud-based license plate recognition service used globally. It supports real-time plate reading and even vehicle analytics. It offers the ability to adapt to local plate formats, including those with non-Latin characters. However, deployment in offline or restricted environments may be challenging.

Sighthound, a commercial LPR product used in traffic and surveillance systems, is designed for high accuracy under various visual conditions. It's typically used in cities or parking facilities where conditions such as lighting, angle, and movement can affect recognition. While highly capable, these systems are typically optimized for Western license plate formats.

Surveillance equipment providers like **Hikvision** and **Dahua** integrate ANPR technology directly into their smart cameras. These are often installed in gates, parking lots, or toll stations. The systems automatically detect plates and send the information to control panels or access systems. Their default configurations typically prioritize common international plate styles, and support for dual-script or Arabic plates like Egyptian ones may be limited or unavailable without deep firmware customization.

In the academic field, several projects like **ALPRNet**, **CR-NET**, and **Deep ANPR** have explored the use of convolutional neural networks (CNNs) for license plate detection and recognition. These projects contributed to the research and accuracy of OCR models and end-to-end systems that can work across a variety of conditions. They have laid the groundwork for custom models that can be trained on local datasets, such as Egyptian car plates, for more regionally precise performance.

Compared to these systems, our project focuses specifically on the Egyptian context by training models on local plates, which include both Arabic characters and digits. Unlike general-purpose commercial products, our system is tailored to handle the specific structure and style of Egyptian car plates. It also integrates directly with a database to provide full details about the vehicle and driver when a recognized plate exists in the system. The real-time capability, low resource requirements, and local database linking make our system a specialized and effective tool for use in universities, secure facilities, and smart gate applications in Egypt.

Chapter 2: Background

2.1 Introduction

In an era of increasing technological advancement, the ability to monitor, detect, and manage vehicular access is essential for both public and private security. Our graduation project addresses this need by developing a car plate recognition system that automatically detects license plates using a live camera feed, logs them with timestamps, and compares the captured plate numbers with a pre-defined list of authorized vehicles. The system stores and manages data effectively using a reliable database. This chapter presents the essential background to understand the project's motivations, technological foundations, and how it connects to real-world applications.

2.2 Problem Statement

Managing access control based on vehicle license plates is a common requirement in secure facilities such as universities, residential compounds, private companies, and parking lots. Traditionally, this task is done manually by guards or gatekeepers, which can be time-consuming and prone to human error. Moreover, with increasing traffic and growing concerns for security and automation, a manual method no longer meets modern efficiency and safety standards. Some common issues with the traditional manual approach include:

- Unauthorized access due to oversight.
- No automatic logging or tracking of car visits.
- Lack of timestamped history for each vehicle.
- Delays at entry/exit points due to verification processes. To

address these issues, an automated system using computer vision and database management becomes necessary.

2.3 Project Goals

The project aims to:

- Provide a system that uses a camera to detect vehicle license plates in real time.
- Recognize Egyptian license plates, including both Arabic characters and digits.
- Extract and log plate numbers with accurate timestamps.
- Store plate details in a structured database.
- Provide CRUD operations for managing allowed cars and detected cars.
- Enable quick access to data such as recent detections, all detections, and authorized vehicle lists.

2.4 Related Technologies

This section outlines the core technologies used in this project.

2.4.1 OpenCV

OpenCV (Open Source Computer Vision Library) is a popular open-source computer vision and machine learning software library. It provides a real-time optimized Computer Vision toolkit and is widely used in image processing tasks such as object detection, face recognition, and more. In this project, OpenCV is used to:

- Access and stream video from the camera.
- Detect and extract license plates from video frames.
- Pre-process plate images before sending them to recognition models.

2.4.2 EasyOCR

EasyOCR is a Python-based Optical Character Recognition (OCR) library that supports multiple languages, including Arabic. It is used in this project to recognize both Arabic letters and digits from Egyptian license plates. It allows for fast and accurate reading of license plate text after OpenCV isolates the relevant area.

2.4.3 MongoDB

MongoDB is a NoSQL, document-based database that provides high flexibility and performance. It is used to:

- Store allowed car data including plate numbers, owner information, and contact details.
- Log detected vehicles along with timestamps.
- Allow dynamic querying and retrieval of detection history and authorization status. MongoDB is ideal for this use case due to:
 - Schema flexibility for storing structured and semi-structured data.
 - Easy integration with Node.js (via Mongoose) or Python (via PyMongo) if needed.

- Scalability and performance for future expansion.

2.5 Egyptian License Plates Recognition

One of the main challenges in this project is recognizing Egyptian vehicle license plates, which combine Arabic letters and numbers. Key points of difficulty include:

- **Arabic OCR complexity:** Arabic letters change shape depending on position in the word, making recognition harder.
- **Mixed language plates:** Egyptian plates often use Arabic letters and Western numbers, requiring multi-language OCR.
- **Plate design variations:** Plate color, layout, and fonts can vary depending on the type of vehicle (private, taxi, transport, etc.).

To address this, we use EasyOCR's multilingual support and custom filtering logic to map recognized characters to standard formats. Preprocessing steps (e.g., grayscale conversion, thresholding, resizing) improve OCR accuracy

2.6 Use Cases

The system can be applied in many real-world

scenarios such as:

- University gates: Automatically detect and log vehicles of students, staff, and visitors.
- Residential communities: Identify and differentiate between residents and guests.
- Corporate campuses: Restrict unauthorized vehicles and maintain an access log.
- parking facilities: Automate ticketless entry/exit based on license plate validation.

By digitizing the car detection and logging process, organizations can reduce cost, enhance efficiency, and improve security

Chapter 3: System Design

3.1 System Features

Our system is divided into three main components, each working together to enable fast, real-time identification of Egyptian car plates and linking them with driver or vehicle data.

1. Detection Section

This part of the system is responsible for capturing the video feed and detecting license plates. A camera is used to continuously monitor the scene in real time. When a vehicle appears, the system identifies the area containing the plate, isolates it from the frame, and prepares it for character recognition.

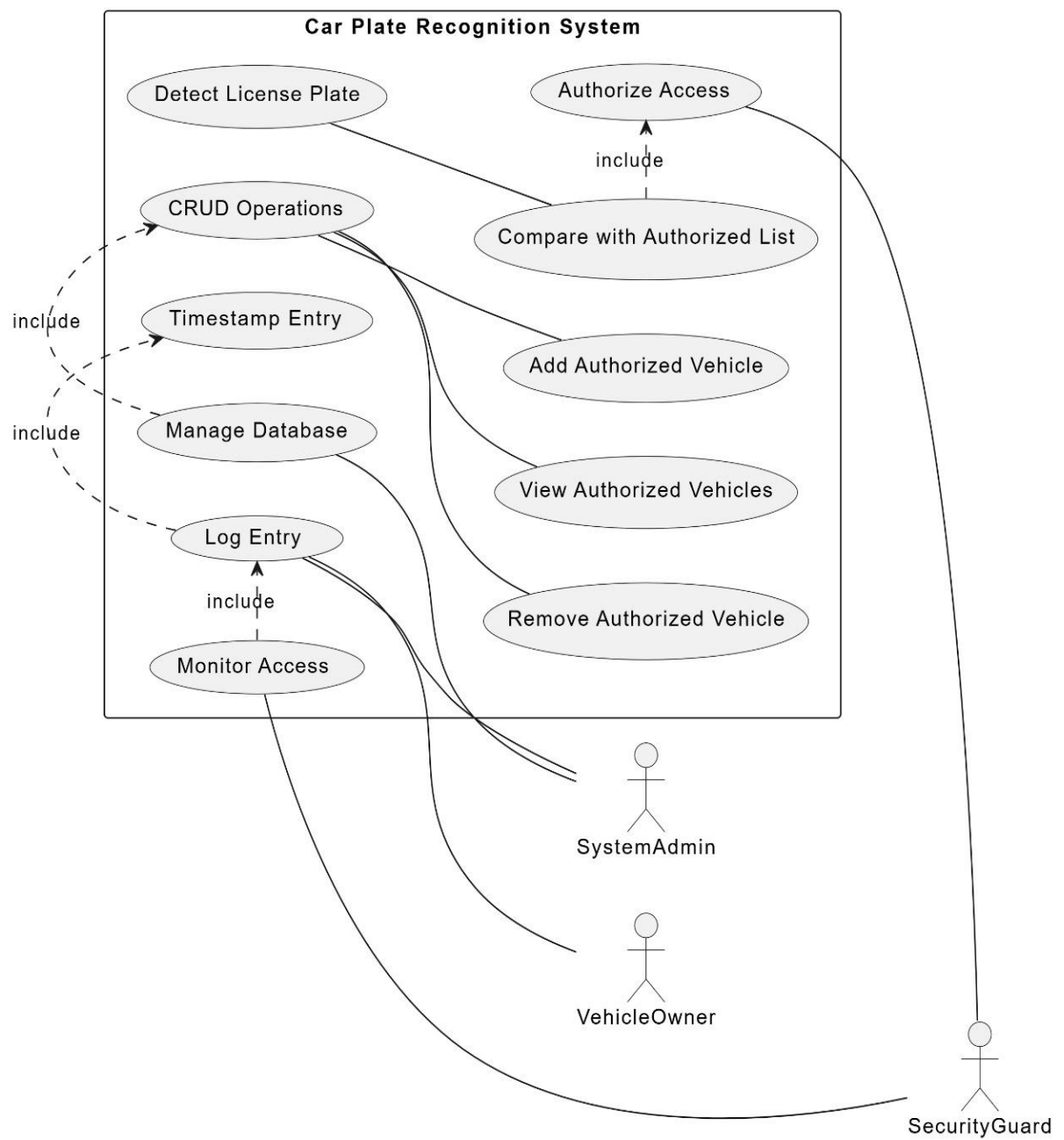
2. Recognition Section

Once the plate is detected, it is passed to the recognition stage. This phase uses a trained model to identify the Arabic letters and digits on the plate. The extracted characters are then reshaped and rendered in the correct order using tools that support the Arabic language. This ensures accurate reading of the full plate number as it would appear to a human observer.

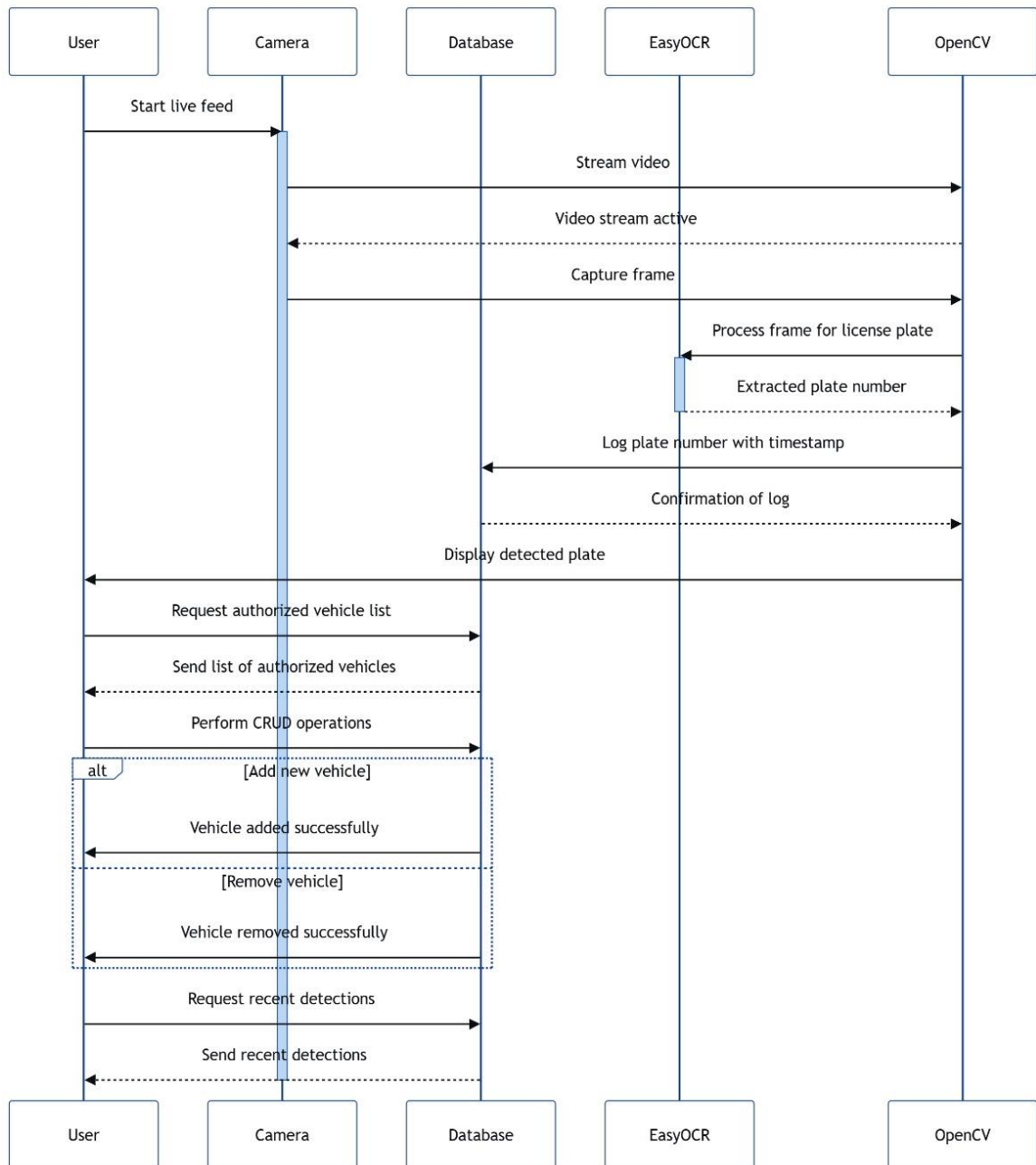
3. Database Interaction Section

After the plate number is recognized, the system automatically checks it against a preloaded database. If the number exists in the database, detailed information about the corresponding vehicle and driver is retrieved. This data is then displayed clearly on the application's interface. This section allows for quick and efficient verification in real-time, without requiring any manual data input.

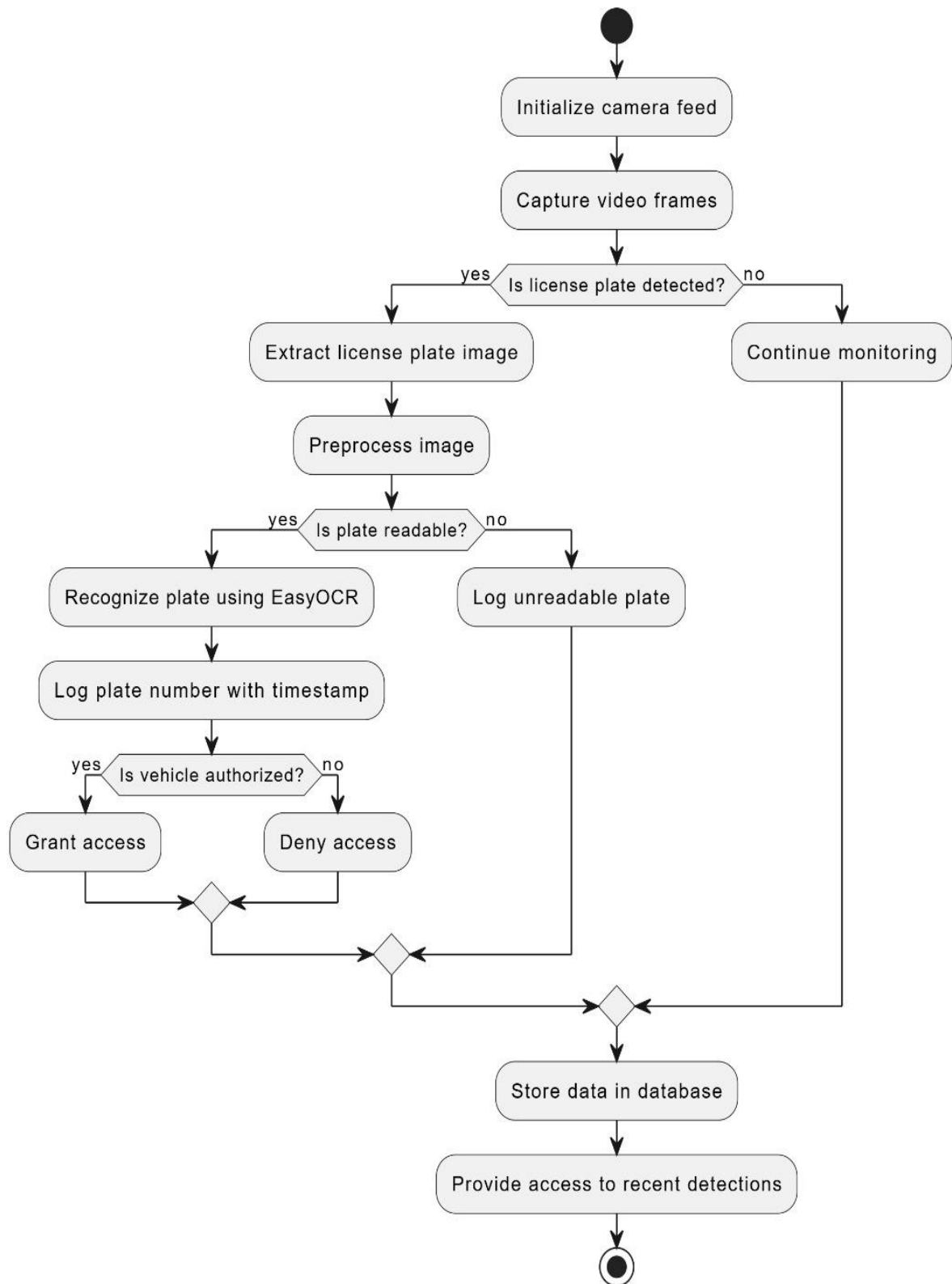
3.2 Usecase Diagram:



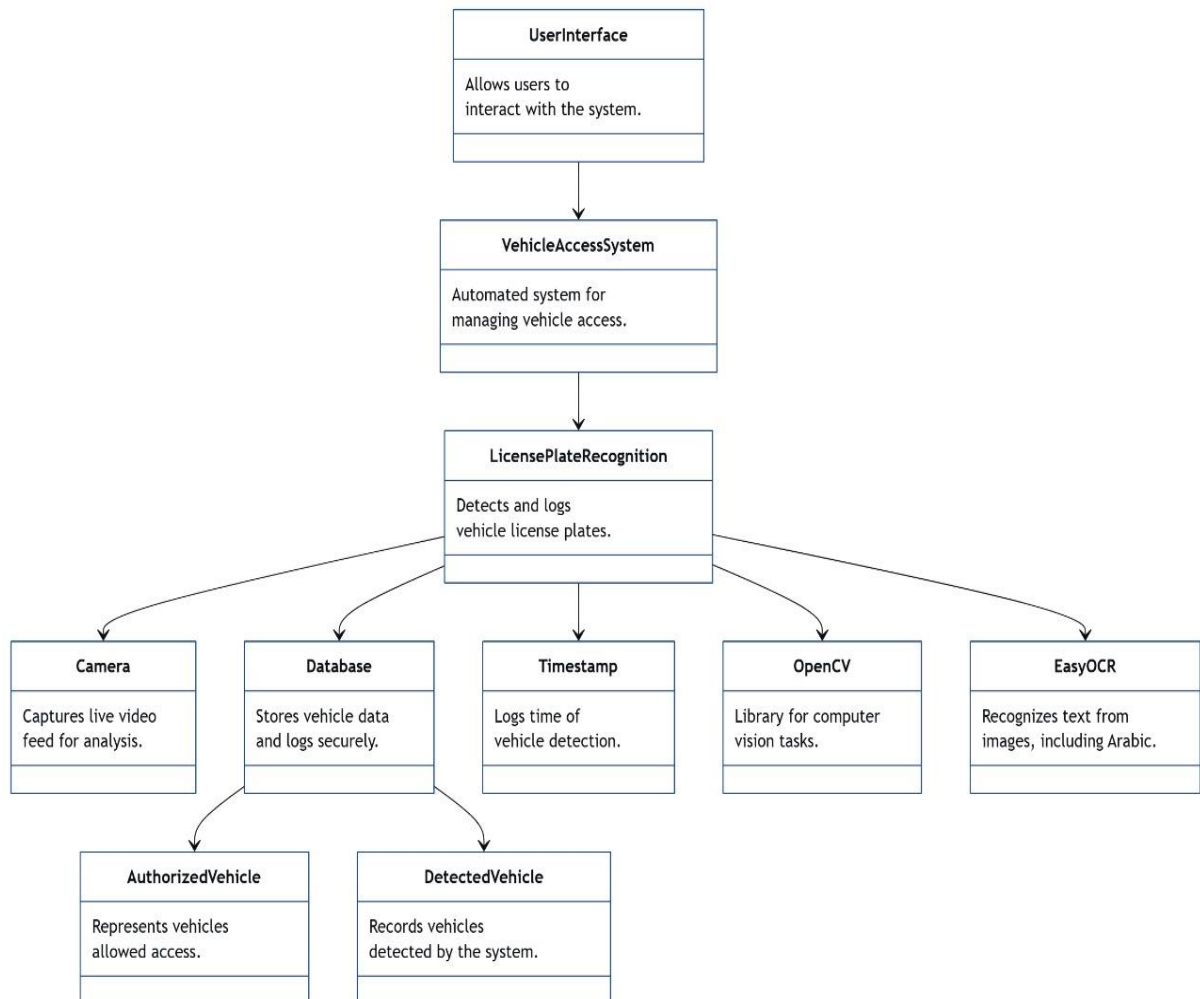
3.3 Sequence Diagram:



3.4 Activity Diagram:

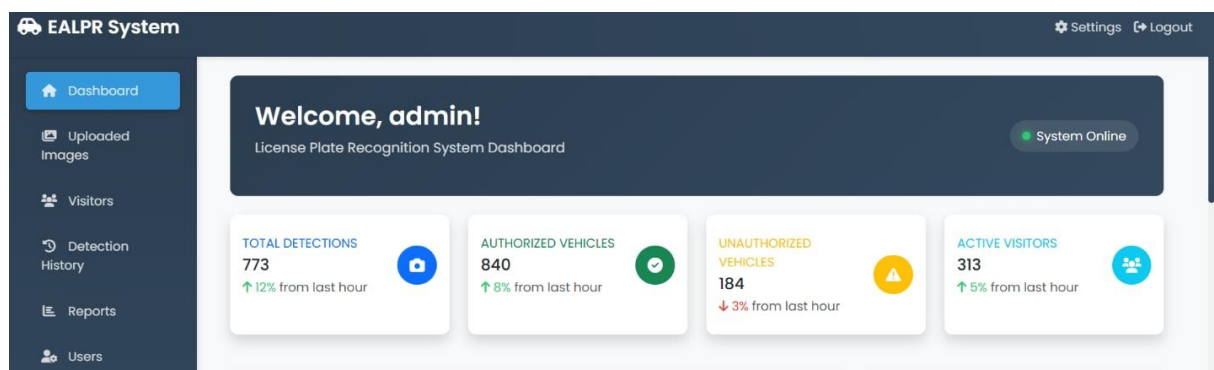
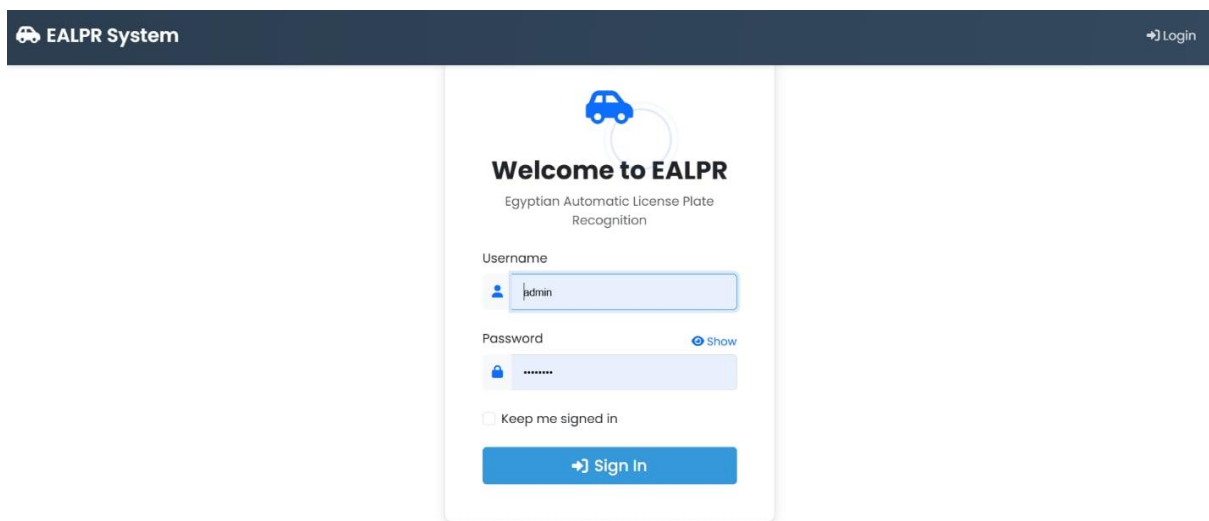


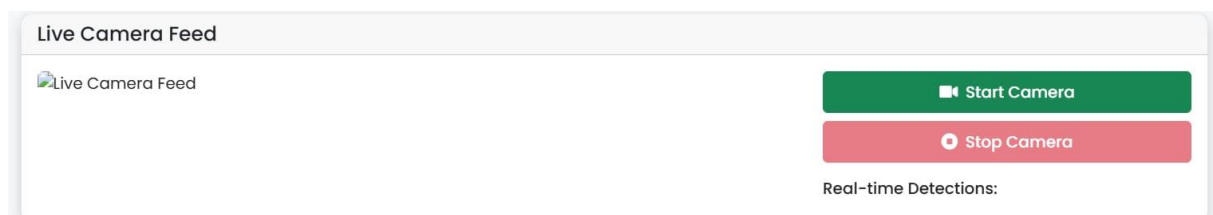
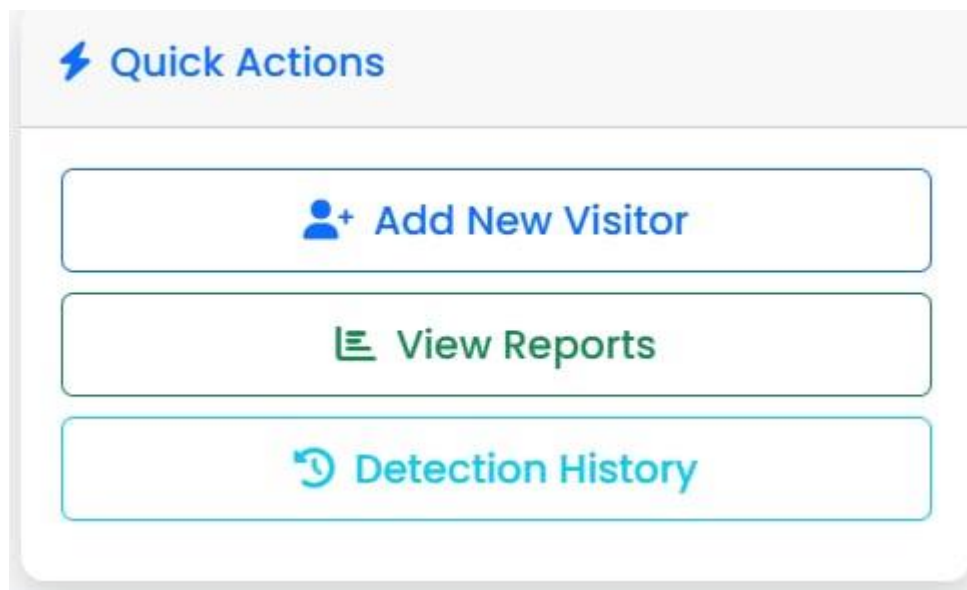
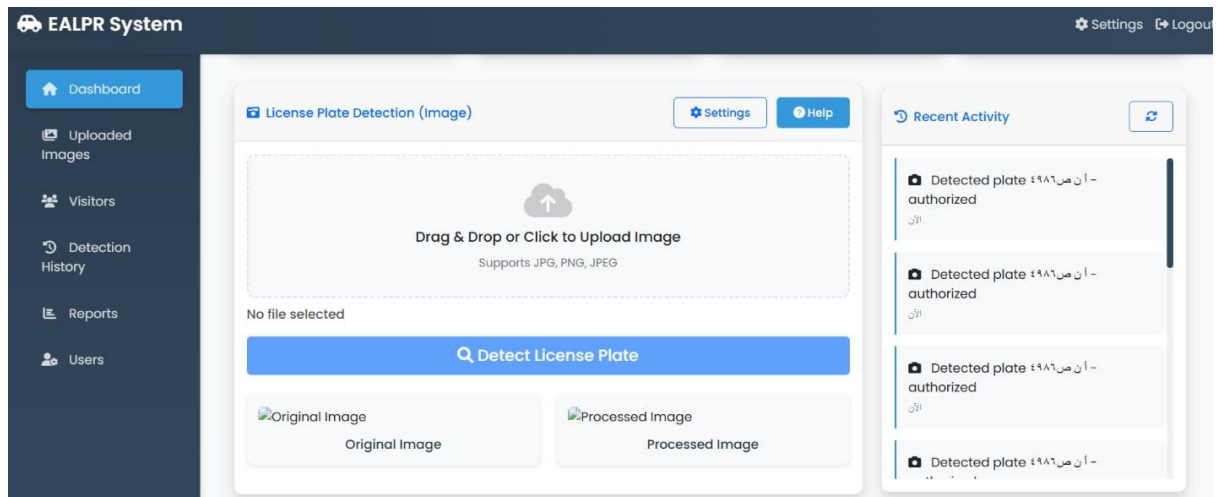
3.5 Class Diagram:



Chapter 4: System Implementation and results

4.1 Overview on implementation steps





Uploaded Images

View all processed license plate images

All Status

All Time

Refresh

Clear

Export

authorized

2025-06-11 21:36

م م م ١٤١٤

عمر محمود

authorized

2025-06-11 18:39

أن ص ٩٨٦٤

احمد علي

authorized

2025-06-11 18:39

أن ص ٩٨٦٤

احمد علي

authorized

2025-06-11 18:39

أن ص ٩٨٦٤

احمد علي

Manage Visitors

Add New Visitor

Name

Visitor Code

License Plate

Status

Pending

Responsible Department

General Department

Add Visitor

Visitor List

No.	Name	Visitor Code	License Plate	Entry Date	Entry Time	Status	Responsible Department	General Department	Actions
1	علي احمد	V1024	أ ر ق ١٧٩٢	2025-06-09	12:03:48	Authorized	Administration	General Affairs	Edit Unauthorize Delete
2	احمد محمود	V1023	ي ق ٨٧٣٧	2025-06-09	11:47:34	Authorized	Administration	General Affairs	Edit Unauthorize Delete
3	علي احمد	V1022	أ ن د ٣١١١	2025-06-07	15:36:31	Authorized	Administration	General Affairs	Edit Unauthorize Delete

Detection Details



Original Image



Processed Image



Plate Number: ١٤١٤ م م م

Date: 2025-06-12

Status: مسجل / مصرح له بالدخول

Time: 12:36 am

Confidence: 88.11%

Visitor Information

Name: عمر محمود

Entry Time: 16:46:40

ID Number: 199

Exit Time: 2025-05-30 19:46:40

License Plate: ١٤١٤ م م م

Department: Human Resources

Detection History

Export Controls

Status Filter

Date Range

Format

Export

Clear Detection History

Recent Detections

Date	Time	Plate Number	Status	Visitor Name	Actions
2025-06-11	21:36:18	١٤١٤ م.م	authorized	عمر محمود	View
2025-06-11	18:39:41	أ ن ص ٩٨٦	authorized	أحمد تلي	View
2025-06-11	18:39:38	أ ن ص ٩٨٦	authorized	أحمد تلي	View
2025-06-11	18:39:33	أ ن ص ٩٨٦	authorized	أحمد تلي	View
2025-06-11	18:39:30	أ ن ص ٩٨٦	authorized	أحمد تلي	View
2025-06-11	18:39:27	أ ن ص ٩٨٦	authorized	أحمد تلي	View
2025-06-11	18:39:24	أ ن ص ٩٨٦	authorized	أحمد تلي	View

Reports and Analytics

Total Visitors

1024

Authorized Visitors

840

Authorization Rate

82.0%

Daily Visitor Traffic

Peak Hours

Export Reports

Daily Report

Weekly Report

Monthly Report

User Management

[+ Add New User](#)

Username	Role	Created At	Actions
admin	Admin	2025-05-30 10:06:08	Edit Refresh
Admin	User	2025-06-02 18:00:07	Edit Delete Refresh

System Settings

[+ Add New Setting](#)

Key	Value	Description	Last Updated	Actions
confidence_threshold	0.5	Minimum confidence threshold for plate detection	2025-05-30 13:06:08	Edit
max_image_size	5242880	Maximum allowed image size in bytes	2025-05-30 13:06:08	Edit
allowed_extensions	jpg,jpeg,png	Allowed image file extensions	2025-05-30 13:06:08	Edit
system_name	EALPR System	Name of the system	2025-05-30 13:06:08	Edit
maintenance_mode	false	System maintenance mode	2025-05-30 13:06:08	Edit

4.2 Future Work

There are several potential enhancements and extensions that can be applied to our car plate recognition system to improve its performance, usability, and adaptability in future versions:

1) Dual-Model Recognition for Improved Accuracy

A future version of the system could include two separate models: one trained on English characters and another trained specifically on Arabic characters. This would allow the system to run both models simultaneously, compare their outputs, and discard the less accurate result. The enhanced recognition process would reduce errors and improve overall accuracy, especially for mixed-script plates such as Egyptian license plates.

2) Mobile Application Integration

Developing a mobile version of the system could make it more practical for use in field scenarios such as security patrols or parking enforcement. A mobile app would allow users to scan car plates using a smartphone camera and instantly receive results, making the system more portable and accessible.

3) Advanced Database Integration

In the current version, working with local files posed challenges in managing plate entries. Future work could involve integrating a structured relational database system (e.g., PostgreSQL, MySQL, or an enhanced SQLite schema) to streamline data management, making it easier to add, delete, update, and search for driver and vehicle records.

4) Multimedia Logging and Video Annotation Support

As an extension, the system could incorporate a logging feature that records plate recognition events into video files, and automatically generates subtitles or transcriptions of recognized data. This can be enhanced further with the integration of an audio transcription engine (e.g., IBM Watson Speech-to-Text) to provide voice annotations or spoken alerts. Subtitles could then be downloaded and added to the video for documentation or auditing purposes. how to copy this to a word file

4.3 Screenshots and Code Sample

Login

```
{% extends "base.html" %}

{% block title %}Login - EALPR System{% endblock %}

{% block content %}
<div class="login-wrapper">
  <div class="login-container">
    <div class="card">
      <div class="card-body p-5">
        <div class="text-center mb-4">
          <div class="logo-wrapper mb-3">
            <i class="fas fa-car-side fa-3x text-primary"></i>
            <div class="logo-rings"></div>
          </div>
          <h1 class="fw-bold h2 mb-2">Welcome to EALPR</h1>
          <p class="text-muted">Egyptian Automatic License Plate Recognition</p>
        </div>

        {% with messages = get_flashed_messages(with_categories=true) %}
          {% if messages %}
            {% for category, message in messages %}
              <div class="alert alert-{{ category }} alert-dismissible fade show">
                <i class="fas fa-{{ 'info-circle' if category == 'info' else 'exclamation-triangle' }}"></i>
                {{ message }}
                <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
              </div>
            {% endfor %}
          {% endif %}
        {% endwith %}

        <form method="POST" action="{{ url_for('login') }}" id="loginForm" class="needs-validation">
          <div class="mb-4">
            <label for="username" class="form-label">Username</label>
            <div class="input-group has-validation">
              <span class="input-group-text bg-light">
                <i class="fas fa-user text-primary"></i>
              </span>
            </div>
          </div>
        </form>
      </div>
    </div>
  </div>
</div>

```

User management

```
{% for user in users %}
<tr>
  <td>{{ user.username }}</td>
  <td>
    <span class="badge {% if user.is_admin %}bg-danger{% else %}bg-primary{% endif %} {{ 'Admin' if user.is_admin else 'User' }}>
  </td>
  <td>{{ user.created_at.strftime('%Y-%m-%d %H:%M:%S') }}</td>
  <td>
    <button class="btn btn-sm btn-info edit-user" data-user-id="{{ user.id | string }}">
      <i class="fas fa-pencil-alt"></i>
    </button>
    {% if user.id != current_user.id %}
    <button class="btn btn-sm btn-danger delete-user" data-user-id="{{ user.id | string }}">
      <i class="fas fa-trash"></i>
    </button>
    {% endif %}
    <button class="btn btn-sm btn-secondary view-activity" data-user-id="{{ user.id | string }}">
      <i class="fas fa-history"></i>
    </button>
  </td>
</tr>
{% endfor %}
```

Live camera feed

```
camera = None

def process_frame_and_detect(frame, frame_count, plate_detection_model, ocr_model, last_detection_time, last_plate_text):
    """Helper function to process frame and emit detection results for both video and live camera feed"""
    # Check frame similarity if we have a previous frame
    if last_frame is not None:
        frame_diff = cv2.absdiff(frame, last_frame)
        change_percent = np.sum(frame_diff > 30) / frame_diff.size * 100
        if change_percent < 1.0: # Less than 1% change
            return None, None, frame

    temp_frame_path = os.path.join(TEMP_DIR, f"frame_{frame_count}.jpg")
    try:
        cv2.imwrite(temp_frame_path, frame)
        processed_img, plate_text, processing_success, error_msg, plate_confidence = process_image(
            temp_frame_path,
            plate_detection_model,
            ocr_model
        )

        if processing_success:
            current_time = datetime.utcnow()

            # Skip duplicate detections if tracking last detection time
            if last_detection_time and plate_text in last_detection_time:
                time_since_last = (current_time - last_detection_time[plate_text]).total_seconds()
                if time_since_last < VIDEO_DUPLICATE_WINDOW:
                    return None, None, frame

            # Look up visitor information
            visitor = Visitor.objects(license_plate=plate_text).first()
            status = 'authorized' if visitor else 'unauthorized'
```

Database

```
def init_database():
    print(" ¶ Connecting to MongoDB...")
    connect(MONGODB_DB_NAME, host=MONGODB_URI)

    try:
        # Create indexes for User collection
        print("Creating User collection and indexes...")
        User.objects._collection.create_index("username", unique=True)
        User.objects._collection.create_index("is_admin")
        User.objects._collection.create_index("last_login")
        print("✓ User collection created")

        # Create indexes for Visitor collection
        print("Creating Visitor collection and indexes...")
        Visitor.objects._collection.create_index("license_plate", unique=True)
        Visitor.objects._collection.create_index("entry_datetime_utc")
        Visitor.objects._collection.create_index("authorized")
        Visitor.objects._collection.create_index("status")
        Visitor.objects._collection.create_index("visitor_code", unique=True)
        print("✓ Visitor collection created")

        # Create indexes for UserActivity collection
        print("Creating UserActivity collection and indexes...")
        UserActivity.objects._collection.create_index("user")
        UserActivity.objects._collection.create_index("timestamp")
        UserActivity.objects._collection.create_index("action")
        print("✓ UserActivity collection created")
```

```
class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(128))
    full_name = db.Column(db.String(100))
    phone = db.Column(db.String(20))
    address = db.Column(db.String(200))
    role = db.Column(db.String(20), default='user')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    last_login = db.Column(db.DateTime)
    is_active = db.Column(db.Boolean, default=True)

class Vehicle(db.Model):
    __tablename__ = 'vehicles'
    id = db.Column(db.Integer, primary_key=True)
    plate_number = db.Column(db.String(20), unique=True, nullable=False)
    make = db.Column(db.String(50))
    model = db.Column(db.String(50))
    color = db.Column(db.String(30))
    owner_id = db.Column(db.Integer, db.ForeignKey('users.id'))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    last_updated = db.Column(db.DateTime, onupdate=datetime.utcnow)
```


Ocr production

```
import os
import argparse
import cv2
import numpy as np
import csv
from PIL import ImageFont, ImageDraw, Image
from tensorflow import keras
from keras.preprocessing.image import load_img
import arabic_reshaper
from bidi.algorithm import get_display
from ultralytics import YOLO
import time
import pandas as pd

# Load YOLO models
ocr_model = YOLO(r'D:\GP\runs\detect\yolo11m_car_plate\weights\best.pt')
plate_detector_model = YOLO(r'D:\GP\runs\detect\yolo_car_plate\weights\best.pt')

# Define font path
font_path = r"D:\GP\alfont_com_arial-1.ttf"

# Define class labels mapping
CLASS_LABELS_MAPPING = {
    0: "ا", 1: "ب", 2: "ت", 3: "ث", 4: "ج", 5: "د", 6: "ذ", 7: "ر",
    8: "ز", 9: "س", 10: "ش", 11: "ط", 12: "ظ", 13: "ع", 14: "ف", 15: "ق",
    16: "ك", 17: "خ", 18: "ح", 19: "غ", 20: "ل", 21: "م", 22: "ن", 23: "هـ",
    24: "و", 25: "ي", 26: "و", 27: "ق", 28: "ج", 29: "ص", 30: "ط", 31: "ث",
    32: "ت", 33: "ث", 34: "ج", 35: "ي", 36: "ذ", 37: "ز"
}

# Define colors for visualization
COLORS = [
    (255, 0, 0), (34, 75, 12), (0, 0, 255), (255, 255, 0),
    (255, 0, 255), (21, 52, 72), (66, 50, 168)
]
```

Config file

```
load_dotenv()

# Base directory
BASE_DIR = Path(__file__).resolve().parent

# Model paths
MODEL_PATH = Path(os.getenv('MODEL_PATH', BASE_DIR / 'working' / 'runs' / 'detect' / 'yolo_car_plate' / 'weights'))
OCR_MODEL_PATH = Path(os.getenv('OCR_MODEL_PATH', BASE_DIR / 'working' / 'runs' / 'detect' / 'yolo11m_car_plate'))

# Ensure model files exist
if not MODEL_PATH.exists():
    raise FileNotFoundError(f"Plate detection model file not found at {MODEL_PATH}")
if not OCR_MODEL_PATH.exists():
    raise FileNotFoundError(f"OCR model file not found at {OCR_MODEL_PATH}")

# Tesseract OCR path (update this based on your installation)
TESSERACT_CMD = 'C:/Program Files/Tesseract-OCR/tesseract.exe'

# Detection settings
CONFIDENCE_THRESHOLD = float(os.getenv('CONFIDENCE_THRESHOLD', '0.5'))
PLATE_CLASS_ID = int(os.getenv('PLATE_CLASS_ID', '0'))
MIN_PLATE_AREA = 1000 # Minimum area for plate detection
MAX_PLATE_AREA = 10000 # Maximum area for plate detection

# OCR settings
OCR_CONFIG = '--psm 6 --oem 3 -l ara' # Optimized for Arabic text

# Processing settings
DRAW_DETECTIONS = True # Whether to draw detections on the frame
SAVE_DETECTIONS = True # Whether to save detected plates
SAVE_DIR = BASE_DIR / 'detected_plates' # Directory to save detected plates
TEMP_DIR = Path(os.getenv('TEMP_DIR', BASE_DIR / 'temp')) # Directory for temporary files
```

```
OCR_CONFIG = '--psm 6 --oem 3 -l ara' # Optimized for Arabic text

# Processing settings
DRAW_DETECTIONS = True # Whether to draw detections on the frame
SAVE_DETECTIONS = True # Whether to save detected plates
SAVE_DIR = BASE_DIR / 'detected_plates' # Directory to save detected plates
TEMP_DIR = Path(os.getenv('TEMP_DIR', BASE_DIR / 'temp')) # Directory for temporary files

# Create necessary directories
SAVE_DIR.mkdir(exist_ok=True)
TEMP_DIR.mkdir(exist_ok=True)

# Image processing settings
ALLOWED_EXTENSIONS = {'jpg', 'jpeg', 'png'} # Supported image formats
ALLOWED_VIDEO_EXTENSIONS = {'mp4', 'avi', 'mov'} # Supported video formats
MAX_IMAGE_SIZE = int(os.getenv('MAX_IMAGE_SIZE', '5242880')) # 5MB
MAX_VIDEO_SIZE = int(os.getenv('MAX_VIDEO_SIZE', '104857600')) # 100MB

# Video processing settings
VIDEO_FRAME_SKIP = int(os.getenv('VIDEO_FRAME_SKIP', '5')) # Process every 5th frame
VIDEO_DUPLICATE_WINDOW = int(os.getenv('VIDEO_DUPLICATE_WINDOW', '30')) # Window for duplicate detect

# OCR enhancement settings
✓ ENHANCEMENT_SETTINGS = {
    'denoise_strength': 30,
    'denoise_template_size': 7,
```


Model training and results

```
result = model_ocr.predict(source=resize_image, conf=0.25)

import matplotlib.pyplot as plt

# Assuming 'result' contains the predictions and the image
predicted_image = result[0].plot() # Plotting the result image

# Display the image using Matplotlib
plt.imshow(predicted_image)
plt.axis('off') # Hide the axis
plt.show()
```

0: 320x640 1 4, 1 7, 1 8, 1 jeem, 1 noon, 2 qaafs, 1 yaa, 165.5ms

Speed: 1.6ms preprocess, 165.5ms inference, 1.9ms postprocess per image at shape (1, 3, 320, 640)



e_version= PP-structurev2)

[2025/05/06 15:06:22] ppocr DEBUG: dt_boxes num : 1, elapsed : 0.060385704040527344

[2025/05/06 15:06:22] ppocr DEBUG: cls num : 1, elapsed : 0.03626561164855957

[2025/05/06 15:06:23] ppocr DEBUG: rec_res num : 1, elapsed : 0.10006594657897949

Detected text: ن ق ق ن with confidence 0.66



[20]: [('0.6550573110580444', 'ن ق ق ن')]

```

fitness: 0.6962119888354716
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([[ 0.39815,  0.53385,  0.61885,  0.68177,  0.61251,  0.62226,  0.65798,  0.68639,  0.66291,  0.67056,  0.6328,  0.68903,
 0.66686,  0.72272,  0.5804,  0.72542,  0.71702,  0.63798,  0.72625,  0.72176,  0.56651,  0.70465,  0.70703,  0.69295,
 0.70282,  0.65733,  0.7097,  0.73495,  0.64008,  0.69637,  0.70103,  0.73738,  0.68158,  0.67804,  0.62997,  0.73132,  0.
60485,  0.68305])
names: {0: '0', 1: '1', 2: '2', 3: '3', 4: '4', 5: '5', 6: '6', 7: '7', 8: '7aa', 9: '8', 10: '9', 11: 'Taa', 12: 'Thaa', 13: 'ain', 14: 'alif', 15: 'baa', 16: 'daad', 1
7: 'daal', 18: 'faa', 19: 'ghayn', 20: 'haa', 21: 'jeem', 22: 'kaaf', 23: 'khaa', 24: 'laam', 25: 'meem', 26: 'noon', 27: 'qaaf', 28: 'raa', 29: 'saad', 30: 'seen', 31:
'sheen', 32: 'taa', 33: 'thaa', 34: 'waw', 35: 'yaa', 36: 'zaal', 37: 'zay'})
plot: True
results_dict: {'metrics/precision(B)': 0.9735669211489458, 'metrics/recall(B)': 0.9887493713529911, 'metrics/mAP50(B)': 0.9877621171934976, 'metrics/mAP50-95(B)': 0.6638
175301290242, 'fitness': 0.6962119888354716}
save_dir: PosixPath('runs/detect/yolo11m_car_plate2')
speed: {'preprocess': 1.0034785650990086, 'inference': 29.710033808053737, 'loss': 0.0003480389219573574, 'postprocess': 1.1683414416115099}
task: 'detect'

```

```

'Recall']]
fitness: 0.9233496530744887
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([[ 0.91543])
names: {0: 'car_plate'}
plot: True
results_dict: {'metrics/precision(B)': 0.9951862764965277, 'metrics/recall(B)': 0.9925079441513395, 'metrics/mAP50(B)': 0.9946694440630918, 'metrics/mAP50-95(B)': 0.9154
252318535328, 'fitness': 0.9233496530744887}
save_dir: PosixPath('runs/detect/yolo_car_plate2')
speed: {'preprocess': 0.3245533241398545, 'inference': 24.571125666500894, 'loss': 0.000339977830062618, 'postprocess': 0.7592415428571885}
task: 'detect'

```

Processing function

```

yolo_model = '/kaggle/input/modelss/pytorch/default/1/best.pt'

resize_image = crop_Plate(yolo_model,
                            "/kaggle/input/egyptian-cars-plates/EALPR Vehicules dataset/Vehicles/0010.jpg")

resize_image

```

image 1/1 /kaggle/input/egyptian-cars-plates/EALPR Vehicules dataset/Vehicles/0010.jpg: 640x288 1 license_plate, 87.3ms
Speed: 8.3ms preprocess, 87.3ms inference, 279.1ms postprocess per image at shape (1, 3, 640, 288)



```

# Function to crop the car plate from the uploaded image
def crop_Plate(yolo_model, img):
    model = YOLO(yolo_model)
    count = 0

    # Perform prediction on the image
    results = model.predict(source=img, conf=0.25)

    # Open the image
    image = Image.open(img)

    for result in results:
        if result.boxes is not None and len(result.boxes) > 0:
            max_width = -1
            selected_box = None

            # Iterate through all detected boxes to find the one with the maximum width
            for box in result.boxes:
                res = box.xyxy[0] # Get the coordinates of the bounding box
                width = res[2].item() - res[0].item() # Calculate width (x_max - x_min)

                if width > max_width:
                    max_width = width
                    selected_box = res # Store the coordinates of the selected box

```

```

# Function to preprocess the input image
def preprocess_image(image_path, target_size=(640, 640)):
    # Read the image
    image = cv2.imread(image_path)

    # Resize image to target size (for YOLO or detection model)
    image_resized = cv2.resize(image, target_size)

    # Normalize the image (convert to float and scale between 0-1)
    image_normalized = image_resized / 255.0

    # Convert image to the format needed by the model (Batch, Height, Width, Channels)
    input_image = np.expand_dims(image_normalized, axis=0) # Add batch dimension

    return input_image, image_resized

```

```

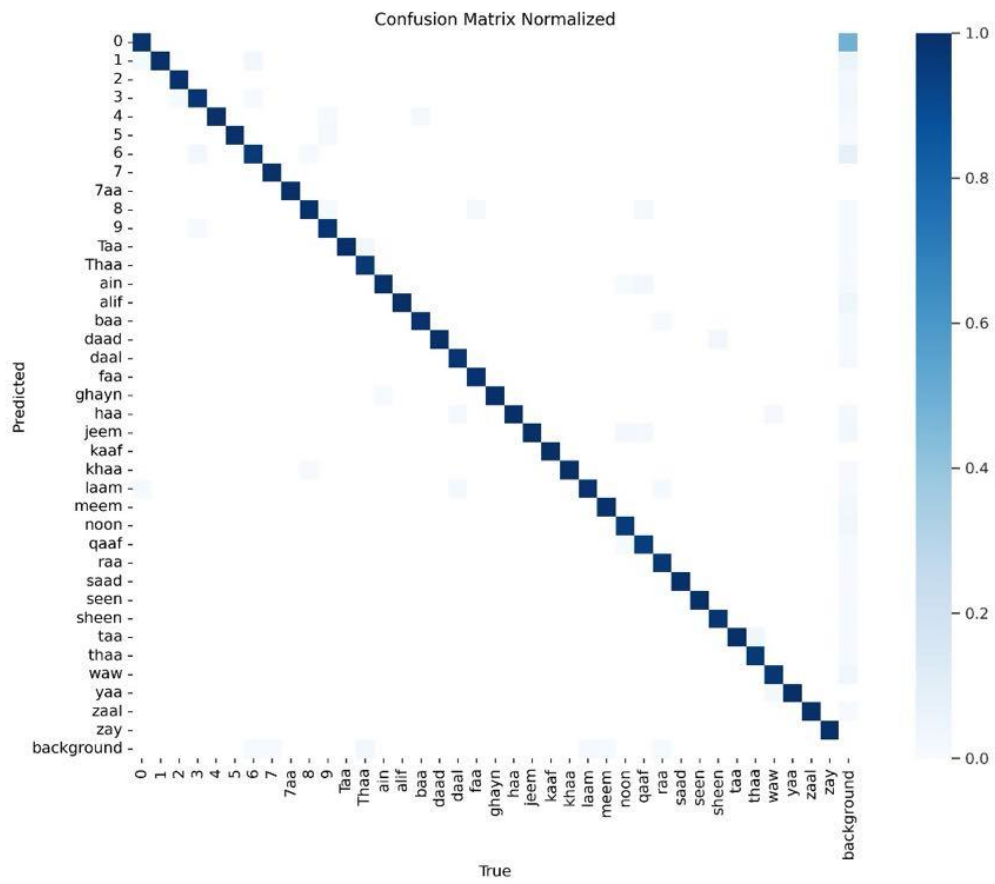
# Function to load bounding boxes from the label file
def load_bounding_boxes(label_file):
    bounding_boxes = []
    with open(label_file, 'r') as file:
        for line in file.readlines():
            values = line.strip().split()
            class_id = int(values[0]) # Class ID (optional if needed)
            x_center = float(values[1])
            y_center = float(values[2])
            width = float(values[3])
            height = float(values[4])
            bounding_boxes.append([x_center, y_center, width, height])
    return bounding_boxes

# Function to draw bounding boxes on the image
def draw_bounding_boxes(image, bounding_boxes):
    h, w = image.shape[:2]
    for bbox in bounding_boxes:
        x_center, y_center, box_width, box_height = bbox
        # Convert YOLO format to corner coordinates
        x1 = int((x_center - box_width / 2) * w)
        y1 = int((y_center - box_height / 2) * h)
        x2 = int((x_center + box_width / 2) * w)
        y2 = int((y_center + box_height / 2) * h)
        # Draw the rectangle on the image
        cv2.rectangle(image, (x1, y1), (x2, y2), (255, 0, 0), 2)

```




Confusion matrix



Bibliography

- 1) <https://docs.ultralytics.com/>
- 2) <https://arxiv.org/abs/1804.02767>
- 3) <https://github.com/tesseract-ocr/tesseract>
- 4) https://thesai.org/Downloads/Volume13No7/Paper_99-Real_time_Egyptian_License_Plate_Detection_and_Recognition.pdf?utm_source=chatgpt.com
- 5) <https://github.com/Mo7amed50/EALPR-SYSTEM>